INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Prof. Jelili O. Oyelade

Topics to be covered

- Concept of Artificial Intelligence
- Problem Solving in Artificial Intelligence (AI)
- Knowledge Representation

CONCEPT OF ARTIFICIAL INTELLIGENCE (AI)

- Since the invention of computers or machines, their capability to perform various tasks went on growing exponentially.
- Humans have developed the power of computer systems in terms of:
  - their diverse working domains,
  - their increasing speed, and
  - reducing size with respect to time.
- A branch of Computer Science named Artificial Intelligence pursues creating the computers or machines as intelligent as human beings.

What is Artificial Intelligence?

- Artificial Intelligence is a way of making a computer, a computer-controlled robot, or a software think intelligently, in the similar manner the intelligent humans think.
- AI is accomplished by studying how human brain thinks and how humans learn, decide, and work while trying to solve a problem,
- the outcomes of this study is then used as a basis of developing intelligent software and systems.

Goals of AI

- **To Create Expert Systems**
  - The systems which exhibit intelligent behavior, learn, demonstrate, explain, and advice its users.
- **To Implement Human Intelligence in Machines**
  - Creating systems that understand, think, learn, and behave like humans.

What Contributes to AI?

- Artificial intelligence is a science and technology based on disciplines such as Computer Science, Biology, Psychology, Linguistics, Mathematics, and Engineering.
- A major thrust of AI is in the development of computer functions associated with human intelligence, such as reasoning, learning, and problem solving.
- Out of the following areas, one or multiple areas can contribute to build an intelligent system.

Programming Without and With AI

- The programming without and with AI is different in following ways

Programming Without AI	**Programming With AI**
A computer program without AI can answer the specific questions it is meant to solve.	A computer program with AI can answer the generic questions it is meant to solve.
Modification in the program leads to change in its structure.	AI programs can absorb new modifications by putting highly independent pieces of information together. Hence you can modify even a minute piece of information of program without affecting its structure.
Modification is not quick and easy. It may lead to affecting the program adversely.	Quick and easy program modification.

Applications of AI

- AI has been dominant in various fields such as:

1. **Gaming** − AI plays crucial role in strategic games such as chess, poker, tic-tac-toe, etc., where machine can think of large number of possible positions based on heuristic knowledge.
2. **Natural Language Processing** − It is possible to interact with the computer that understands natural language spoken by humans.
3. **Expert Systems** − There are some applications which integrate machine, software, and special information to impart reasoning and advising. They provide explanation and advice to the users.
4. **Vision Systems** − These systems understand, interpret, and comprehend visual input on the computer. For example:
   - A spying airplane takes photographs, which are used to figure out spatial information or map of the areas.
   - Doctors use clinical expert system to diagnose the patient

Applications of AI

5. **Speech Recognition** − Some intelligent systems are capable of hearing and comprehending the language in terms of sentences and their meanings while a human talks to it.
   - It can handle different accents, slang words, noise in the background, change in human's noise due to cold, etc.
6. **Handwriting Recognition** − The handwriting recognition software reads the text written on paper by a pen or on screen by a stylus. It can recognize the shapes of the letters and convert it into editable text.
7. **Intelligent Robots** − Robots are able to perform the tasks given by a human.
   - They have sensors to detect physical data from the real world such as light, heat, temperature, movement, sound, bump, and pressure.
   - They have efficient processors, multiple sensors and huge memory, to exhibit intelligence.
   - In addition, they are capable of learning from their mistakes and they can adapt to the new environment.

Intelligent System

- An intelligent system is the ability of a system to calculate, reason, perceive relationships and analogies, learn from experience, store and retrieve information from memory, solve problems, comprehend complex ideas, use natural language fluently, classify, generalize, and adapt new situations.

Types of Intelligence

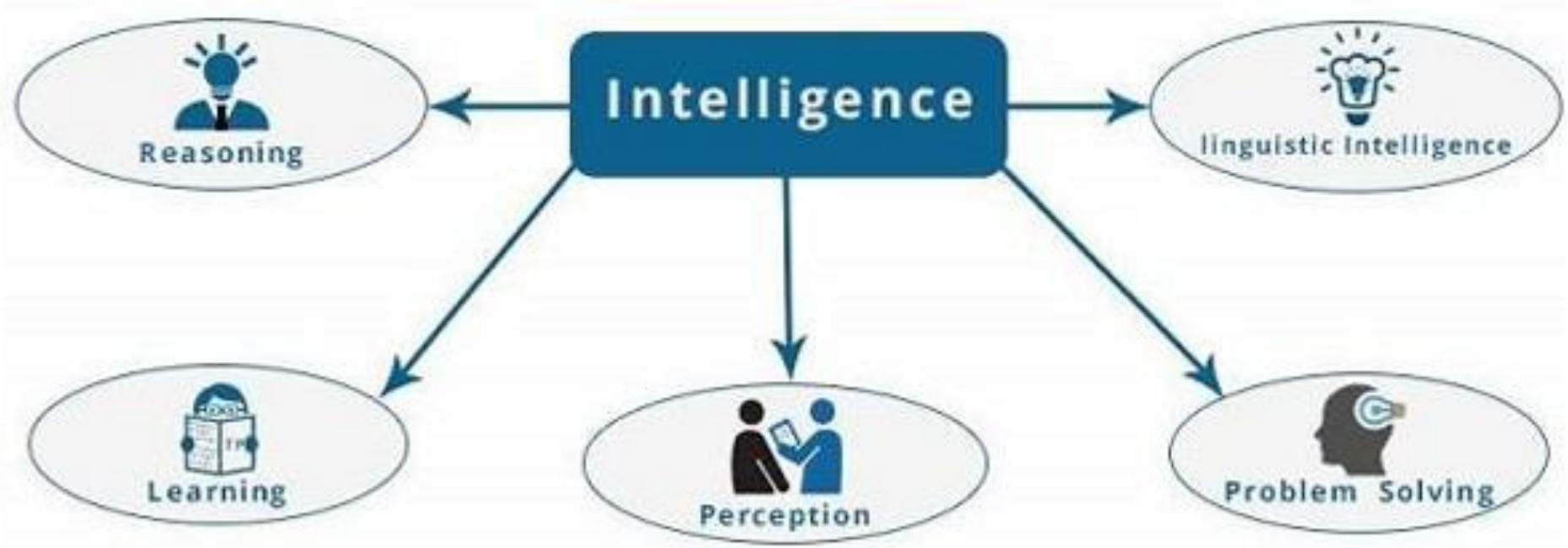
Intelligence	**Description**	**Example**
Linguistic intelligence	The ability to speak, recognize, and use mechanisms of phonology (speech sounds), syntax (grammar), and semantics (meaning).	Narrators, Orators
Musical intelligence	The ability to create, communicate with, and understand meanings made of sound, understanding of pitch, rhythm.	Musicians, Singers, Composers
Logical-mathematical intelligence	The ability of use and understand relationships in the absence of action or objects. Understanding complex and abstract ideas.	Mathematicians, Scientists
Spatial intelligence	The ability to perceive visual or spatial information, change it, and re-create visual images without reference to the objects, construct 3D images, and to move and rotate them.	Map readers, Astronauts, Physicists

Types of Intelligence

Intelligence	**Description**	**Example**
Bodily-Kinesthetic intelligence	The ability to use complete or part of the body to solve problems or fashion products, control over fine and coarse motor skills, and manipulate the objects.	Players, Dancers
Intra-personal intelligence	The ability to distinguish among one's own feelings, intentions, and motivations.	Gautam Buddhha
Interpersonal intelligence	The ability to recognize and make distinctions among other people's feelings, beliefs, and intentions.	Mass Communicators, Interviewers

Components of Intelligence

- It is composed of Reasoning, Learning, Problem Solving, Perception, Linguistic Intelligence



Areas of Artificial Intelligence

- Expert Systems. Examples − Flight-tracking systems, Clinical systems.
- Natural Language Processing, Examples: Google Now feature, speech recognition, Automatic voice output.
- Neural Networks. Examples − Pattern recognition systems such as face recognition, character recognition, handwriting recognition.
- Robotics. Examples − Industrial robots for moving, spraying, painting, precision checking, drilling, cleaning, coating, carving, etc.
- Fuzzy Logic Systems. Examples − Consumer electronics, automobiles, etc.

2. PROBLEM SOLVING IN ARTIFICIAL INTELIGENCE (AI)

- Problem-solving in AI includes various techniques like efficient algorithms, heuristics, and performing root cause analysis to obtain desirable solutions.
- For a given problem in AI, there may be various solutions by different heuristics and at the same time, there are problems with unique solutions.
- These all depend on the nature of the problem to be solved.

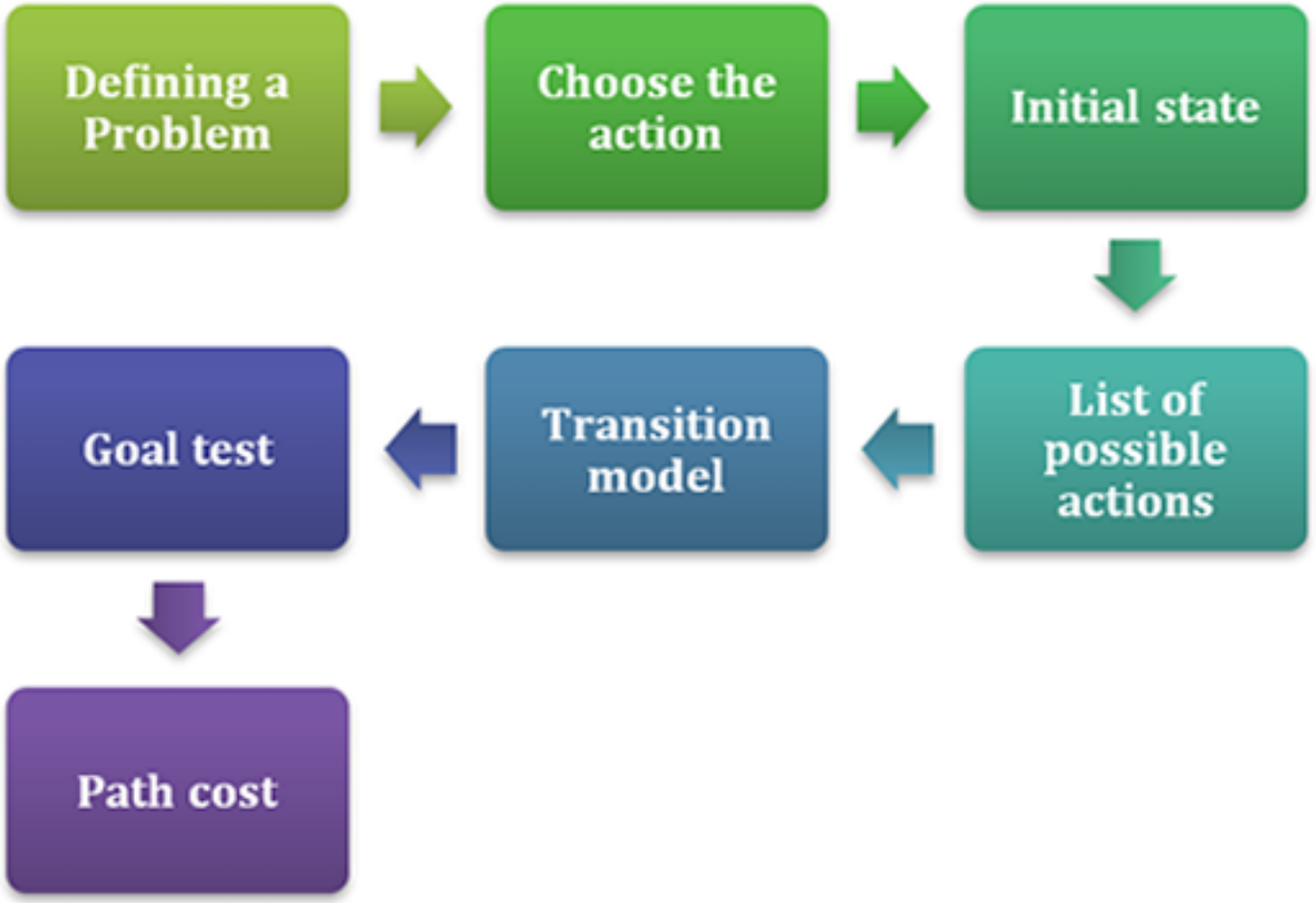
Characteristics of Problems in AI

- Each problem given to an AI is with different aspects of representation and explanation.
- The given problem has to be analyzed along with several dimensions to choose the most acceptable method to solve.
- Some of the key features of a problem are listed below.
  - Can the problem decompose into sub problems?
  - Can any of the solution steps be ignored?
  - Is the given problem universally predictable?
  - Can we select a good solution for the given problem without comparing it to all the possible solutions?
  - Whether the desired output is a state of the world or a path to a state?
  - Did the problem require a large amount of knowledge to solve?
  - Does it require any interaction with computers and humans?

Characteristics of Problems in AI

- Problems in AI can basically be divided into two types. Toy Problems and Real-World Problems.
- Toy Problem: It can also be called a puzzle-like problem which can be used as a way to explain a more general problem-solving technique.
  - For testing and demonstrating methodologies, or to compare the performance of different algorithms, Toy problems can be used.
  - Toy problems are often useful in providing divination about specific phenomena in complicated problems.
  - Large complicated problems are divided into many smaller toy problems that can be understood in detail easily. Sliding-block puzzles, N-Queens problem, Tower of Hanoi are some examples.
- Real-World Problem: As with the name, it is a problem based on the real world. Real-world problems require a solution.
  - It doesn't depend on descriptions, but with a general formulation.
  - Online shopping, Fraud detection, Medical diagnosis are some examples of real-world problems in AI.

Steps performed in problem-solving



Steps performed in problem-solving

- **Goal formulation**: The first step in problem-solving is to identify the problem. It involves selecting the steps to formulate the perfect goal out of multiple goals and selecting actions to achieve the goal.
- **Problem formulation**: The most important step in problem-solving is choosing the action to be taken to achieve the goal formulated.
- **Initial State**: The starting state of the agent towards the goal.
- **Actions**: The list of the possible actions available to the agent.
- **Transition Model**: What each action does is described.
- **Goal Test**: The given state is tested whether it is a goal state.
- **Path cost**: A numeric cost to each path that follows the goal is assigned. It reflects its performance. Solution with the lowest path cost is the optimal solution.

Example for the problem-solving procedure

- Eight queens puzzle
- **Problem**: The problem here is to place eight chess queens on an 8*8 chessboard in a way that no queens will threaten the other. If two queens will come in the same row, column, or diagonal one will attack another.
- Consider the Incremental formulation for this problem: It starts from an empty state and the operator expands a queen at each step.
- Following are the steps involved in this formulation:
  - States: Arranging 0 to 8 queens on the chessboard.
  - Initial State: An empty chessboard
  - Actions: Adding a queen to any of the empty boxes on the chessboard.
  - Transition model: Returns the new state of the chessboard with the queen added in a box.
  - Goal test: Checks whether 8-queens are kept on the chessboard without any possibility of attack.
  - Path cost: Path costs are neglected because only final states are counted.

Problem-solving methods in Artificial Intelligence

1. **Algorithms**: A problem-solving algorithm can be said as a procedure that is guaranteed to solve if its steps are strictly followed.
   - Example: A person wants to find a book on display among the vast collections of the library. He does not know where the book is kept. By tracing a sequential examination of every book displayed in every rack of the library, the person will eventually find the book. But the approach will consume a considerable amount of time. Thus, an algorithmic approach will succeed but are often slow.

- Types of AI Algorithms

  - Regression Algorithms.
  - Instance-Based Algorithms
  - Decision Tree Algorithms
  - Clustering Algorithms
  - Association Rule Learning Algorithms
  - Artificial Neural Network Algorithms
  - Deep Learning Algorithms
  - Search Algorithms

2. **Heuristics:** A problem-solving heuristic can be said as an informal, ideational, impulsive procedure that leads to the desired solution in some cases only. The fact is that the outcome of a heuristic operation is unpredictable. Using a heuristic approach may be more or less effective than using an algorithm.

3. **Root Cause Analysis:** Like the name itself, it is the process of identifying the root cause of the problem. The root cause of the problem is analyzed to identify the appropriate solutions. A collection of principles, techniques, and methodologies are used to identify the root causes of a problem. RCA can identify an issue in the first place.

- The first goal of RCA is to identify the root cause of the problem. The second goal is to understand how to fix the underlying issues within the root cause. The third goal is to prevent future issues or to repeat the success.
- The core principles of RCA are:
  - Focus on correcting the root causes.
  - Treating symptoms for short-term relief.
  - There can be multiple root causes.
  - Focus on HOW? and WHY?
  - Root cause claims are kept back up with concrete cause-effect.
  - Provides information to get a corrective course of action.
  - Analyze how to avoid a root cause in the future.

Problem Solving by Search Strategy

- In AI, the universal technique for problem-solving is searching.
- For a given problem, it is needed to think of all possible ways to attain the existing goal state from its initial state.
- Searching in AI can be defined as a process of finding the solution for a given set of problems.
- A searching strategy is used to specify which path is to be selected to reach the solution.

Terminologies in Search Algorithms

- **Search**: A step-by-step procedure to solve a given search problem. Three main factors of a search problem:
  - Search Space: The set of possible solutions a system may have.
  - Start state: The state where the agent starts the search.
  - Goal test: A function that monitors the current state and returns whether the goal is achieved or not.
- **Search Tree**: The representation of the search problem in the form of a tree. The root of the tree is the initial state of the problem.
- **Actions**: The description of all actions to the agent.
- **Transition model**: Used to represent the description of the action.
- **Solution**: Sequence of actions that leads from the initial node to the final node.
- **Path Cost**: A function that assigns a numeric cost to each path.
- **Optimal Solution**: The solution with the lowest cost.
- **Problem Space**: The environment in which the search takes place.
- **Depth of a problem**: Length of the shortest path from the initial state to the final state.

Properties of Search Algorithms

- There are four essential properties for search algorithms to compare the efficiencies.

1. Completeness: If the algorithm guarantees to return a solution, if there exists at least one for any random input then the algorithm can be said to be complete.

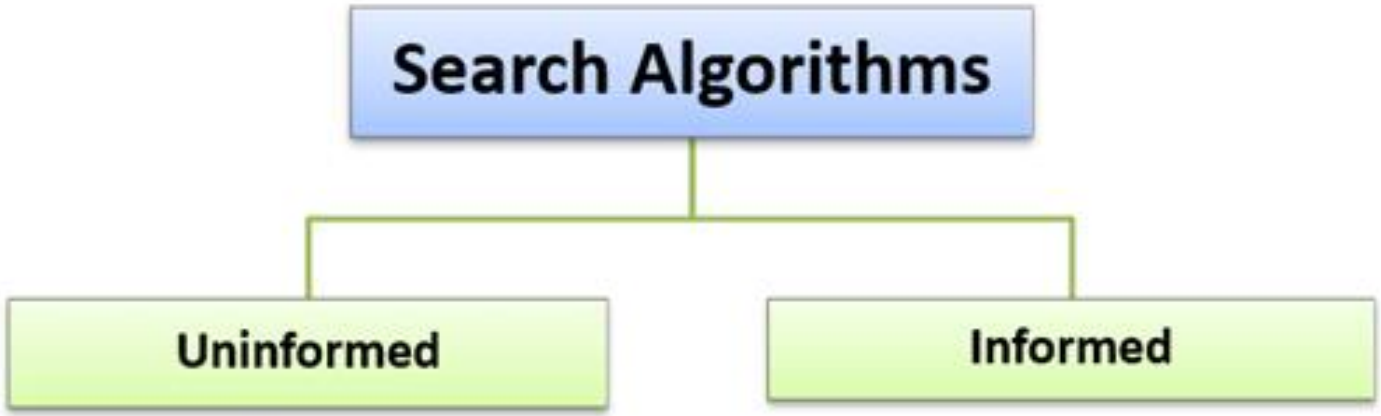
2. Optimality: If the algorithm found the best solution (lowest path cost) among the others, then that solution can be said as the optimal solution. The ability to find the optimal solution is called optimality.

3. Time complexity: The time is taken by the algorithm to complete a task

4. Space Complexity: the required storage space at any point during the search.

Classifications of Search Algorithms

- Search algorithms can be classified mainly into two based on the search problems;



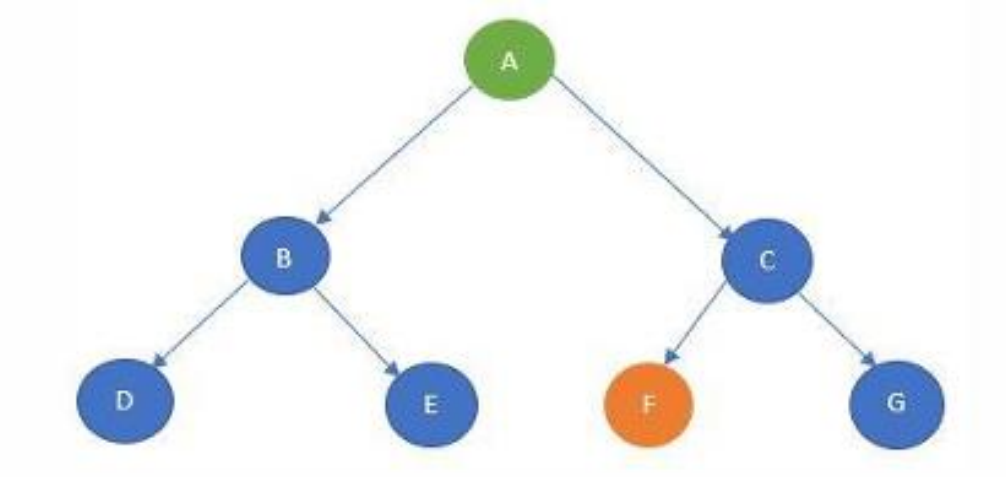
Uninformed Search Algorithms

- The uninformed search algorithm does not have any domain knowledge such as closeness, location of the goal state, etc.
- It behaves in a brute-force way.
- It only knows the information about how to traverse the given tree and how to find the goal state.
- This algorithm is also known as the Blind search algorithm or Brute -Force algorithm.
- The uninformed search strategies are of six types.

  - Breadth-first search
  - Depth-first search
  - Depth-limited search
  - Iterative deepening depth-first search
  - Bidirectional search
  - Uniform cost search

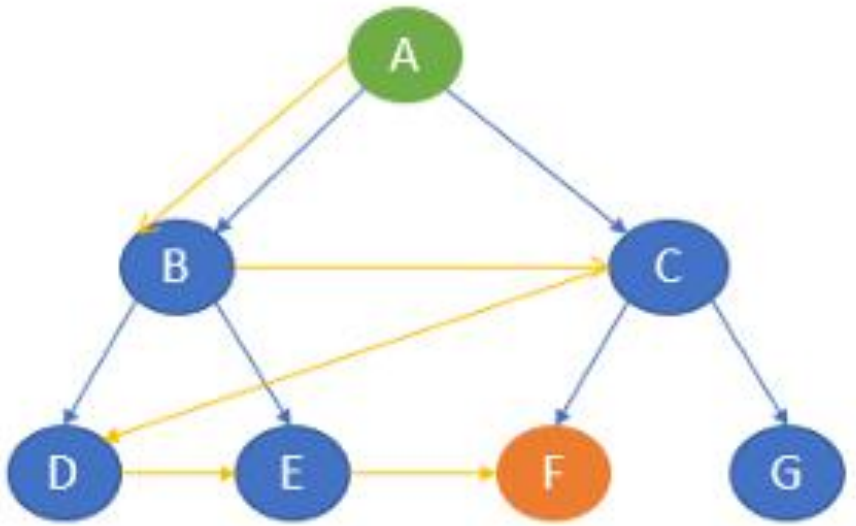
1. Breadth-first search

- It generally starts from the root node and examines the neighbor nodes and then moves to the next level.
- It uses First-in First-out (FIFO) strategy as it gives the shortest path to achieving the solution.
- The BFS is used where the given problem is very small and space complexity is not considered. Now, consider the following tree.



Let’s take node A as the start state and node F as the goal state. The BFS algorithm starts with the start state and then goes to the next level and visits the node until it reaches the goal state.

- In this example, it starts from A and then travel to the next level and visits B and C and then travel to the next level and visits D, E, F and G. Here, the goal state is defined as F. So, the traversal will stop at F.

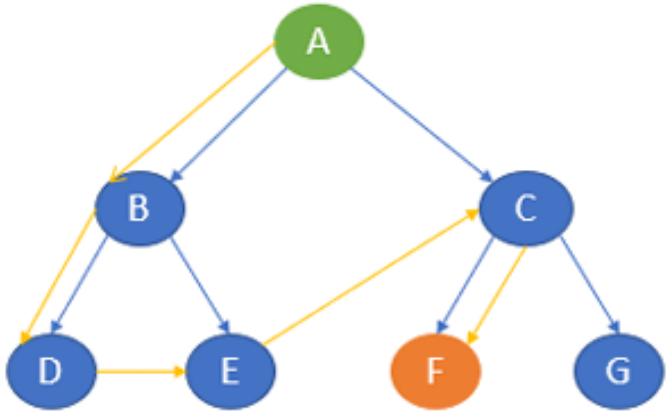


The path of traversal is:

A —-> B —-> C —-> D —-> E —-> F

2. Depth-first search

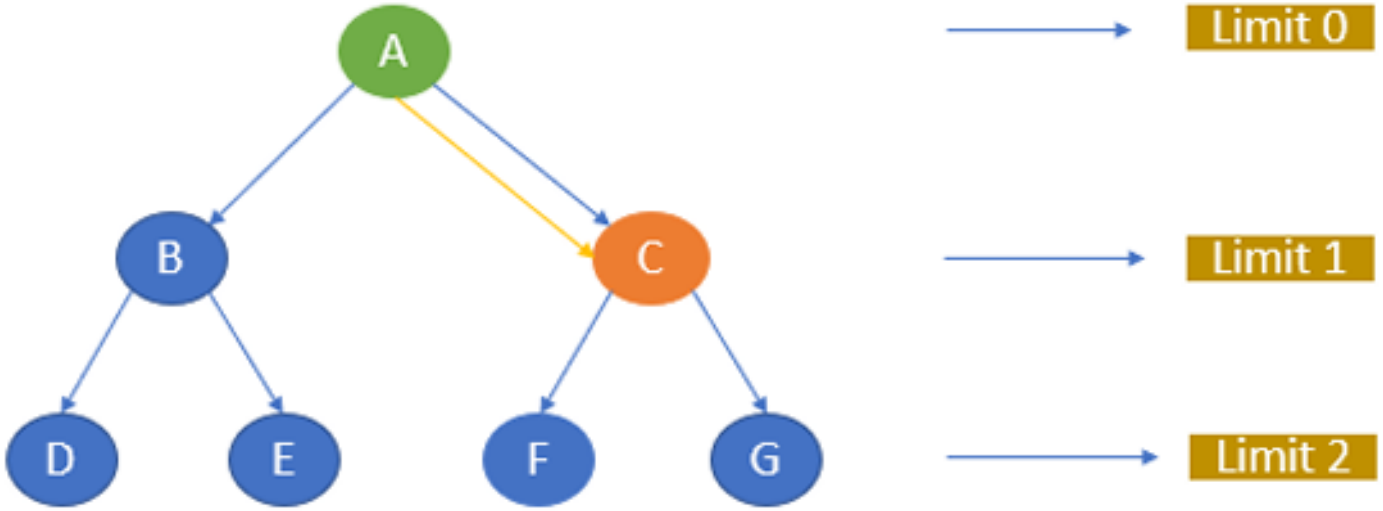
- The depth-first search uses Last-in, First-out (LIFO) strategy and hence it can be implemented by using stack. DFS uses backtracking. That is, it starts from the initial state and explores each path to its greatest depth before it moves to the next path.
- DFS will follow: Root node —-> Left node —-> Right node
- Now, consider the same example tree mentioned in BFS.
  - Here, it starts from the start state A and then travels to B and then it goes to D. After reaching D, it backtracks to B. B is already visited, hence it goes to the next depth E and then backtracks to B. as it is already visited, it goes back to A. A is already visited. So, it goes to C and then to F. F is our goal state and it stops there.
  - The path of traversal is: A--->B--->D--->E--->C--->F



3. **Depth-limited search**: Depth-limited works similarly to depth-first search.

- The difference here is that depth-limited search has a pre-defined limit up to which it can traverse the nodes.
- Depth-limited search solves one of the drawbacks of DFS as it does not go to an infinite path. DLS ends its traversal if any of the following conditions exits.

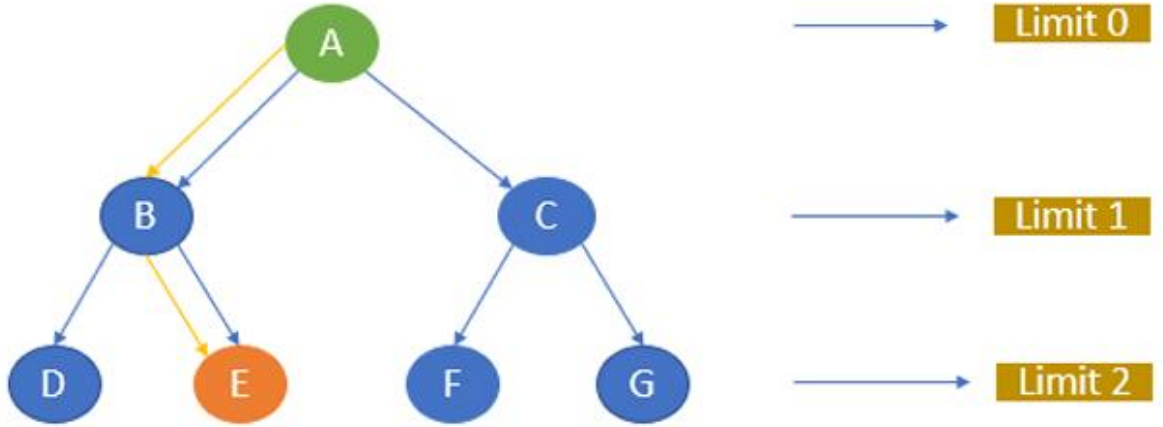
- Standard Failure
  - It denotes that the given problem does not have any solutions.
- Cut off Failure Value
  - It indicates that there is no solution for the problem within the given limit.
- Now, consider the same example. Let's take A as the start node and C as the goal state and limit as 1. The traversal first starts with node A and then goes to the next level 1 and the goal state C is there. It stops the traversal.



- The path of traversal is: A —-> C
- If we give C as the goal node and the limit as 0, the algorithm will not return any path as the goal node is not available within the given limit.
- If we give the goal node as F and limit as 2, the path will be A, C, F.

4. Iterative deepening depth-first:

- Iterative deepening depth-first search is a combination of depth-first search and breadth-first search.
- IDDFS find the best depth limit by gradually adding the limit until the defined goal state is reached.
- With the same example tree; Consider, A as the start node and E as the goal node. Let the maximum depth be 2.
- The algorithm starts with A and goes to the next level and searches for E. If not found, it goes to the next level and finds E.

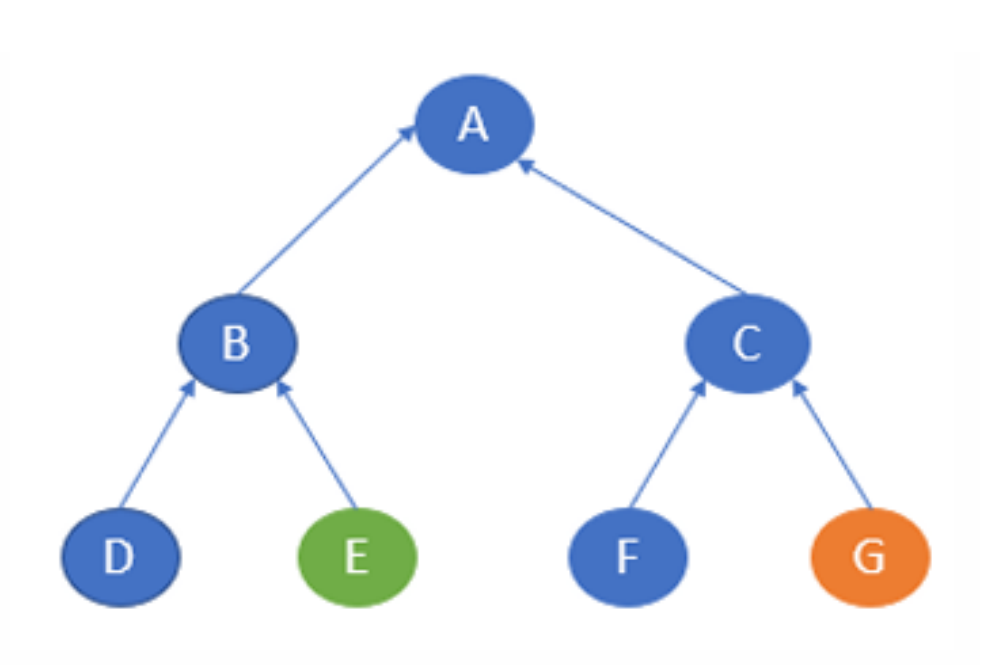


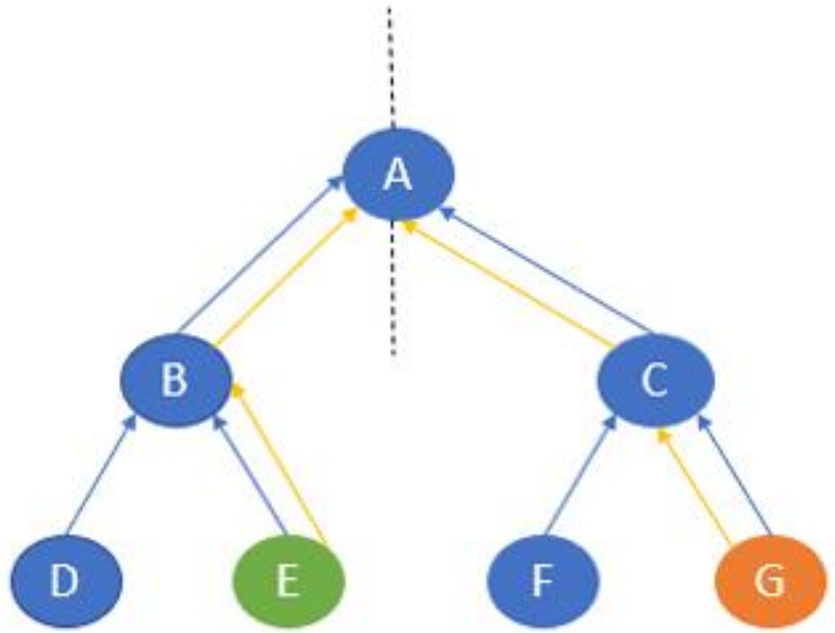
The path of traversal is A —-> B —-> E

5. **Bidirectional search**

- The bidirectional search algorithm is completely different from all other search strategies.
- It executes two simultaneous searches called forward-search and backwards-search and reaches the goal state.
- Here, the graph is divided into two smaller sub-graphs. In one graph, the search is started from the initial start state and in the other graph, the search is started from the goal state.
- When these two nodes intersect each other, the search will be terminated.
- Bidirectional search requires both start and goal start to be well defined and the branching factor to be the same in the two directions. Consider the following graph.

- Here, the start state is E and the goal state is G. In one sub-graph, the search starts from E and in the other, the search starts from G. E will go to B and then A. G will go to C and then A. Here, both the traversal meets at A and hence the traversal ends.

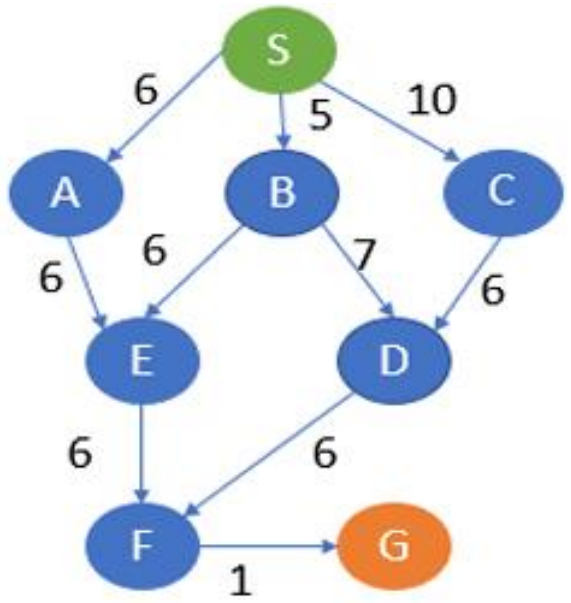




The path of traversal is E —-> B —-> A —-> C —-> G

6. Uniform cost search

- Uniform cost search is considered the best search algorithm for a weighted graph or graph with costs.
- It searches the graph by giving maximum priority to the lowest cumulative cost.
- Uniform cost search can be implemented using a priority queue.
- Consider the below graph where each node has a pre-defined cost.



Here, S is the start node and G is the goal node.
From S, G can be reached in the following ways.

- S, A, E, F, G -> 19
- S, B, E, F, G -> 18
- S, B, D, F, G -> 19
- S, C, D, F, G -> 23

Here, the path with the least cost is S, B, E, F, G.

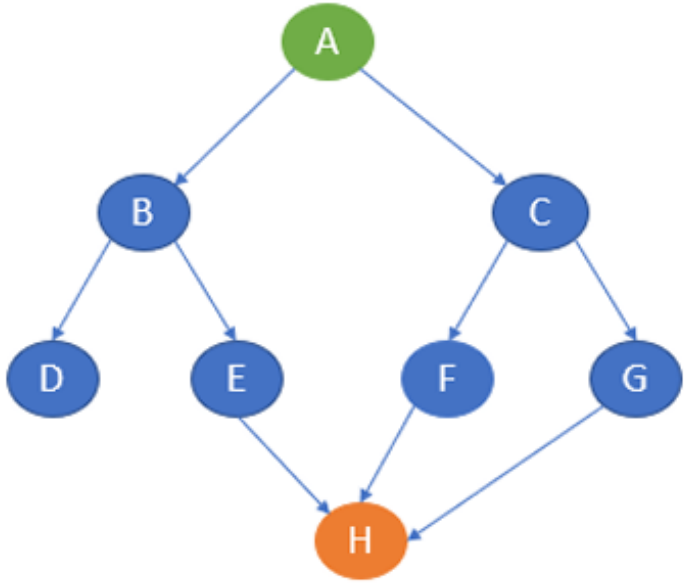
Informed search algorithms

- The informed search algorithm is also called heuristic search or directed search.
- In contrast to uninformed search algorithms, informed search algorithms require details such as distance to reach the goal, steps to reach the goal, cost of the paths which makes this algorithm more efficient.
- Here, the goal state can be achieved by using the heuristic function.
- The heuristic function is used to achieve the goal state with the lowest cost possible.
- This function estimates how close a state is to the goal.

Informed search strategies

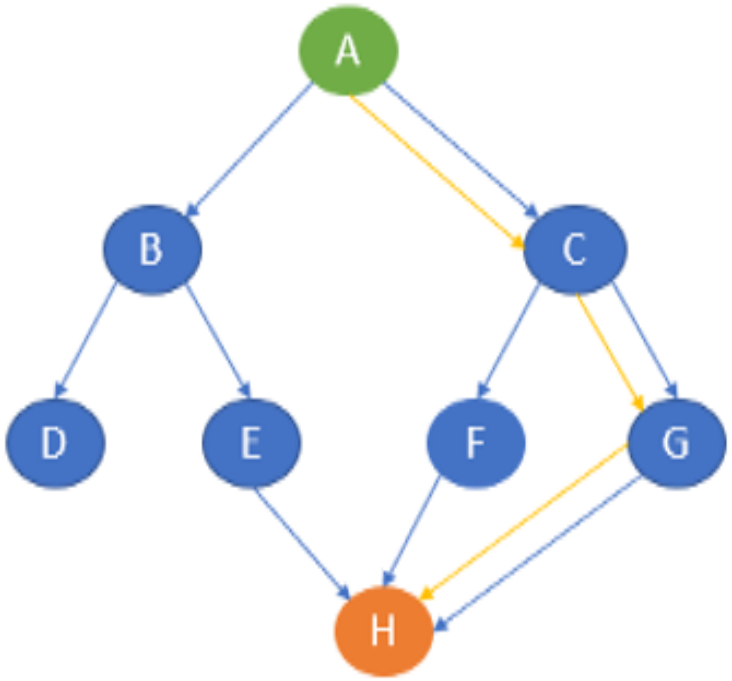
1. Greedy best-first search algorithm

- Greedy best-first search uses the properties of both depth-first search and breadth-first search.
- Greedy best-first search traverses the node by selecting the path which appears best at the moment.
- The closest path is selected by using the heuristic function.
- Consider the graph with the heuristic values.



NODES	HEURISTICS
A	13
B	12
C	4
D	7
E	3
F	8
G	2
H	0

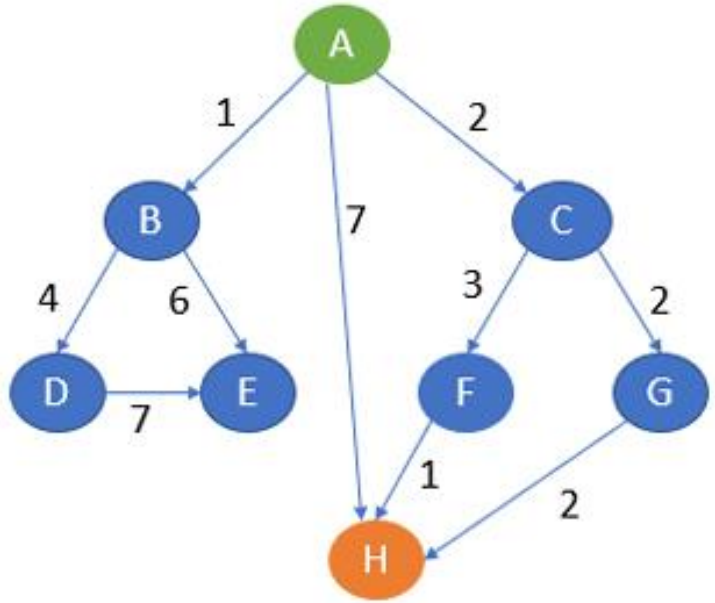
- Here, A is the start node and H is the goal node.
- Greedy best-first search first starts with A and then examines the next neighbour B and C.
- Here, the heuristics of B is 12 and C is 4.
- The best path at the moment is C and hence it goes to C.
- From C, it explores the neighbours F and G.
- The heuristics of F is 8 and G is 2.
- Hence it goes to G. From G, it goes to H whose heuristic is 0 which is also our goal state.



The path of traversal is
A —-> C —-> G —-> H

2. A* search algorithm

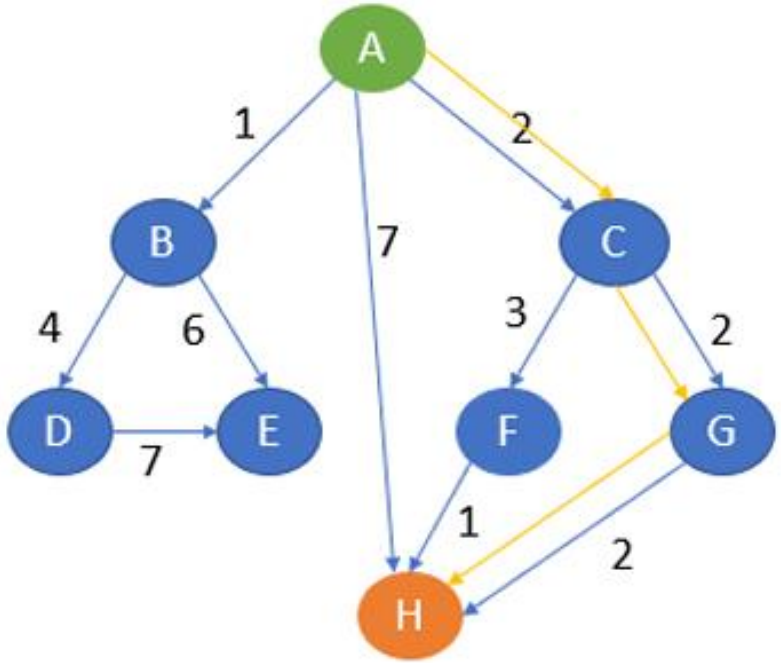
- A* search algorithm is a combination of both uniform cost search and greedy best-first search algorithms.
- It uses the advantages of both with better memory usage.
- It uses a heuristic function to find the shortest path.
- A* search algorithm uses the sum of both the cost and heuristic of the node to find the best path.
- Consider the following graph with the heuristics values as follows.



NODES	HEURISTICS
A	5
B	3
C	4
D	2
E	6
F	3
G	1
H	0

- Let A be the start node and H be the goal node.
- First, the algorithm will start with A. From A, it can go to B, C, H.
- Note the point that A* search uses the sum of path cost and heuristics value to determine the path.
- Here, from A to B, the sum of cost and heuristics is 1 + 3 = 4.
- From A to C, it is 2 + 4 = 6.
- From A to H, it is 7 + 0 = 7.
- Here, the lowest cost is 4 and the path A to B is chosen. The other paths will be on hold. Now, from B, it can go to D or E.
- From A to B to D, the cost is 1 + 4 + 2 = 7.
- From A to B to E, it is 1 + 6 + 6 = 13.

- The lowest cost is 7. Path A to B to D is chosen and compared with other paths which are on hold.
- Here, path A to C is of less cost. That is 6.
- Hence, A to C is chosen and other paths are kept on hold.
- From C, it can now go to F or G.
- From A to C to F, the cost is 2 + 3 + 3 = 8.
- From A to C to G, the cost is 2 + 2 + 1 = 5.
- The lowest cost is 5 which is also lesser than other paths which are on hold. Hence, path A to G is chosen.
- From G, it can go to H whose cost is 2 + 2 + 2 + 0 = 6.
- Here, 6 is lesser than other paths cost which is on hold.
- Also, H is our goal state. The algorithm will terminate here.



The path of traversal is
A —-> C —-> G —-> H

KNOWLEDGE REPRESENTATION

- Much intelligent behavior is based on the use of knowledge; humans spend a third of their useful lives becoming educated.
- There is not yet a clear understanding of how the brain represents knowledge
- **Knowledge representation (KR)** is an area in artificial intelligence that is concerned with how to formally "think", that is, how to use a symbol system to represent "a domain of discourse" that which can be talked about, along with functions that may or may not be within the domain of discourse that allow inference (formalized reasoning) about the objects within the domain of discourse to occur.

Representation and Reasoning System

- Knowledge representation is the study of how knowledge about the world can be represented and what kinds of reasoning can be done with that knowledge.
- In order to use knowledge and reason with it, you need what we call a representation and reasoning system (RRS).
- A representation and reasoning system is composed of a language to communicate with a computer, a way to assign meaning to the language, and procedures to compute answers given input in the language.
- An RRS lets you tell the computer something in a language where you have some meaning associated with the sentences in the language, you can ask the computer questions, and the computer will produce answers that you can interpret according to the meaning associated with the language

Issues in Knowledge Representation

- There are several important issues in knowledge representation:
  - how knowledge is stored;
  - how knowledge that is applicable to the current problem can be retrieved;
  - how reasoning can be performed to derive information that is implied by existing knowledge but not stored directly.
- The storage and reasoning mechanisms are usually closely coupled.
- It is necessary to represent the computer's knowledge of the world by some kind of data structures in the machine's memory.
- Traditional computer programs deal with large amounts of data that are structured in simple and uniform ways.
- A.I. programs need to deal with complex relationships, reflecting the complexity of the real world.

- Typical problem solving (and hence many AI) tasks can be commonly reduced to:
- Representation of input and output data as symbols in a physical symbol
- Reasoning by processing symbol structures, resulting in other symbol structures.

Knowledge Representation in AI Systems

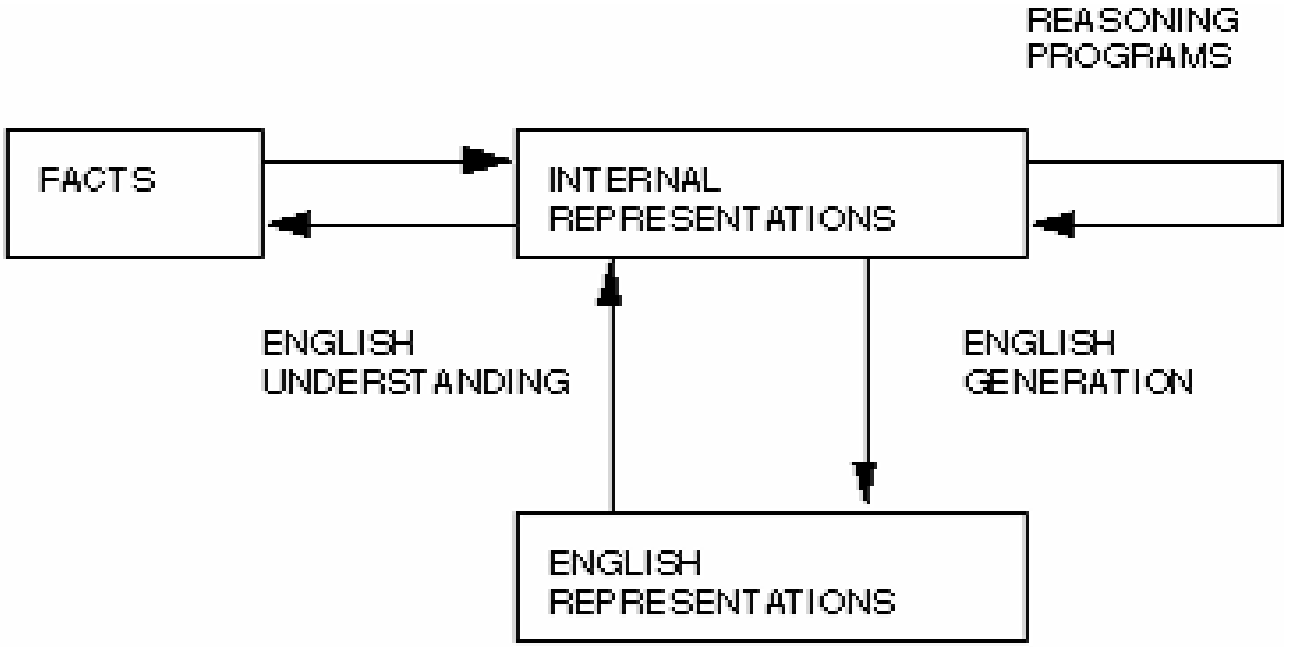
- Some problems highlight search whilst others knowledge representation.
- Several kinds of knowledge might need to be represented in AI systems:
  - **Objects**: Facts about objects in our world domain. e.g. Guitars have strings, trumpets are brass instruments.
  - **Events:** Actions that occur in our world. e.g. Steve Vai played the guitar in Frank Zappa's Band.
  - **Performance:** A behavior like playing the guitar involves knowledge about how to do things.
  - **Meta-knowledge:** knowledge about what we know. e.g. Bobrow's Robot who plan's a trip. It knows that it can read street signs along the way to find out where it is.

Entities in Knowledge Representation

- Thus in solving problems in AI we must represent knowledge and there are two entities to deal with:
- **Facts:** truths about the real world and what we represent. This can be regarded as the knowledge level
- **Representation of the facts:** which we manipulate. This can be regarded as the symbol level since we usually define the representation in terms of symbols that can be manipulated by programs.

Entities in Knowledge Representation

- We can structure these entities at two levels:
- **The knowledge level**: at which facts are described
- **The symbol level**: at which representations of objects are defined in terms of symbols that can
- be manipulated in programs



Using Knowledge

- Let us consider a little further to what applications and how knowledge may be used.
- **Learning:** It refers to acquiring knowledge. This is more than simply adding new facts to a knowledge base. New data may have to be classified prior to storage for easy retrieval, etc.. Interaction and inference with existing facts to avoid redundancy and replication in the knowledge and also so that facts can be updated.
- **Retrieval:** The representation scheme used can have a critical effect on the efficiency of the method. Humans are very good at it.
- **Reasoning:** Infer facts from existing data.

Properties for Knowledge Representation Systems

- The following properties should be possessed by a knowledge representation system:
- **Representational Adequacy:** the ability to represent the required knowledge;
- **Inferential Adequacy:** the ability to manipulate the knowledge represented to produce new knowledge corresponding to that inferred from the original;
- **Inferential Efficiency:** the ability to direct the inferential mechanisms into the most productive directions by storing appropriate guides;
- **Acquisitional Efficiency:** the ability to acquire new knowledge using automatic methods wherever possible rather than reliance on human intervention.

Approaches to Knowledge Representation

1. Simple relational knowledge

- The simplest way of storing facts is to use a relational method where each fact about a set of objects is set out systematically in columns.
- This representation gives little opportunity for inference, but it can be used as the knowledge basis for inference engines.
  - Simple way to store facts.
  - Each fact about a set of objects is set out systematically in columns (Fig below)
  - Little opportunity for inference.
  - Knowledge basis for inference engines.

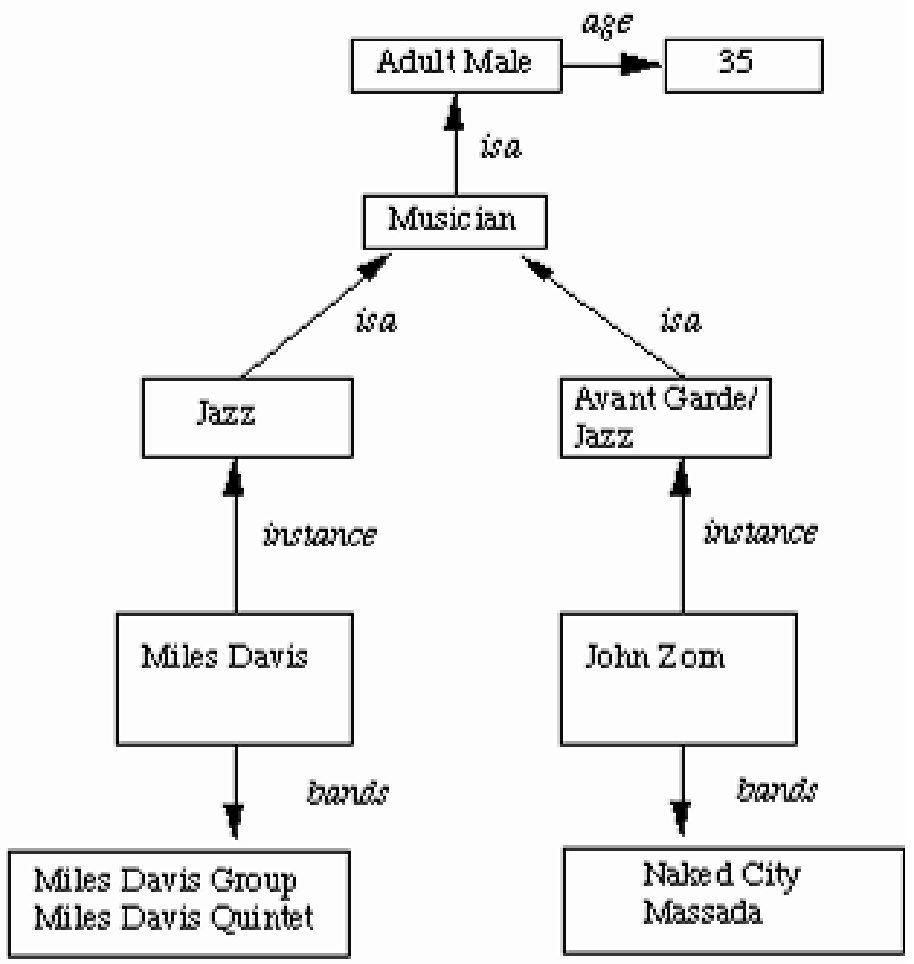
We can ask things like: Who is dead? Who plays Jazz/Trumpet etc.? This sort of representation is popular in database systems.

Musician	Style	Instrument	Age
Miles Davis	Jazz	Trumpet	deceased
John Zorn	Avant Garde	Saxophone	35
Frank Zappa	Rock	Guitar	deceased
John Mclaughlin	Jazz	Guitar	47

Approaches to Knowledge Representation

2. Inheritable knowledge

- Relational knowledge is made up of objects consisting of attributes and corresponding associated values.
- Inheritable knowledge extends the base more by allowing inference mechanisms:
  - Property inheritance
    - Elements inherit values from being members of a class.
    - data must be organized into a hierarchy of classes as shown in the figure below



Approaches to Knowledge Representation

3. Inferential Knowledge

- Represent knowledge as formal logic: All dogs have tails

$$\forall x: \; dog(x) \rightarrow hasatail(x)$$

- Advantages:
  - A set of strict rules.
    - Can be used to derive more facts.
    - Truths of new statements can be verified.
    - Guaranteed correctness.
  - Many inference procedures available to in implement standard rules of logic.
  - Popular in AI systems. e.g Automated theorem proving.

Approaches to Knowledge Representation

4. Procedural Knowledge

- Knowledge encoded in some procedures
  - small programs that know how to do specific things, how to proceed.
  - e.g a parser in a natural language understander has the knowledge that a noun phrase may contain articles, adjectives and nouns.
  - It is represented by calls to routines that know how to process articles, adjectives and nouns.

Advantages:

- Heuristic or domain specific knowledge can be represented.
- Extended logical inferences, such as default reasoning facilitated.
- Side effects of actions may be modelled. Some rules may become false in time. Keeping track of this in large systems may be tricky.

Disadvantages:

- Completeness -- not all cases may be represented.
- Consistency -- not all deductions may be correct.

Issue in Knowledge Representation

- Below are listed issues that should be raised when using a knowledge representation technique:
- **Important Attributes**
  - Are there any attributes that occur in many different types of problem?
  - There are two instance and isa and each is important because each supports property inheritance.
- **Relationships**
  - What about the relationship between the attributes of an object, such as, inverses, existence, techniques for reasoning about values and single valued attributes
- **Granularity**
  - At what level should the knowledge be represented and what are the primitives.

4. Expert System

- Expert system is programs that attempt to perform the duty of an expert in the problem domain in which it is defined.
- Expert systems are computer programs that have been constructed (with the assistance of human experts) in such a way that they are capable of functioning at the standard of (and sometimes even at a higher standard than) human experts in given fields that embody a depth and richness of knowledge that permit them to perform at the level of an expert.

Rule based system

- Using a set of assertions, which collectively form the ‘working memory’, and a set of rules that specify how to act on the assertion set, a rule-based system can be created.
- Rule-based systems are fairly simplistic, consisting of little more than a set of if-then statements, but provide the basis for so-called “expert systems” which are widely used in many fields.
- The concept of an expert system is this: the knowledge of an expert is encoded into the rule set.
- When exposed to the same data, the expert system will perform in a similar manner as the expert.

Element of rule based Expert System

- Rule-based systems are a relatively simple model that can be adapted to any number of problems.
- To create a rule-based system for a given problem, you must have (or create) the following:
  - A set of facts to represent the initial working memory. This should be anything relevant to the beginning state of the system.
  - A set of rules. This should encompass any and all actions that should be taken within the scope of a problem, but nothing irrelevant. The number of rules in the system can affect its performance, so you don't want any that aren't needed.
  - A condition that determines that a solution has been found or that none exists. This is necessary to terminate some rule-based systems that find themselves in infinite loops otherwise.
- In fact, there are three essential components to a fully functional rule based expert system: **the knowledge base, the working memory and the inference engine**.

The knowledge base.

- The knowledge based is the store in which the knowledge in the particular domain is kept.
- The knowledge base stores information about the subject domain.
- However, this goes further than a passive collection of records in a database.
- Rather it contains symbolic representations of experts' knowledge, including definitions of domain terms, interconnections of component entities, and cause-effect relationships between these components.
- The knowledge in the knowledge based is expressed as a collection of fact and rule.
- Each fact expresses relationship between two or more object in the problem domain and can be expressed in term of predicates
- IF condition THEN conclusion where the condition or conclusion are fact or sets of fact connected by the logical connectives NOT, AND, OR.
- Note that we need to create a variable name list to help to deal with long clumsy name and to help writing the rule.

2. The working memory

- The working memory is a temporal store that hold the fact produced during processing and possibly awaiting further processing produced by the Inference engine during its activities.
- Note that the working memory contains only facts and these fact are those produced during the searching process.

3. The inference engine.

- The core of any expert system is its inference engine.
- This is the part of expert system that manipulates the knowledge based to produce new fact in order to solve the given problem.
- An inference engine consists of search and reasoning procedures to enable the system to find solutions, and, if necessary, provide justifications for its answers.
- In this process it can used either forward or backward searching as a direction of search while applying some searching technique such as depth first search, breath first search etc.

- The roles of inference engine are:
- It identified the rule to be fired.
  - The rule selected is the one whose conditional part is the same as the fact been considered in the case of forward chaining or the one whose conclusion part is the one as the fact been considered in the case of backward chaining.
- It resolve conflict when more than one rule satisfy the matching this is called conflict resolution which is based on certain criteria mentioned further.
- It recognizes the goal state. When the goal state is reached it report the conclusion of searching.